Sampled-prior + iterative seeding

`iter_R4_grow_on_sampled_uniform_M5`

Mean LDDT 0.250 → 0.356 (+42.8%)

on the FoldBench-10 train set with MarinFold 1B

Two ideas, one pipeline:

(1) sample contacts honestly with grammar-constrained decoding

(2) iterate the model's own confident contact picks with a growing budget

Recap: the problem and the bars

MarinFold 1B is a Llama-arch LLM. The naive baseline reads distogram bins with zero context:

```
prompt = <begin_sequence> <AAs> <begin_statements>
for (i, j) in all pairs:
    tail = <distance> <p_i> <p_j> <atom_i> <atom_j>
    distance_dist[i, j] = softmax(model(prompt + tail))[<d_*>]
```

- Baseline mean LDDT on 10 train proteins: **0.2496**.
- Original target (issue): mean LDDT ≥ 0.2746 (**+10%**). Cleared early by sharpening + naive seeding.
- Raised target (mid-experiment): mean LDDT ≥ 0.3744 (**+50%**). The hard one.
- Wall-clock budget: $\leq 5\times$ baseline (≤ 6920 s on one A100).
- GT-oracle (cheating, diagnostic only): 0.7167 — the model IS capable; bottleneck is honest contact prediction.

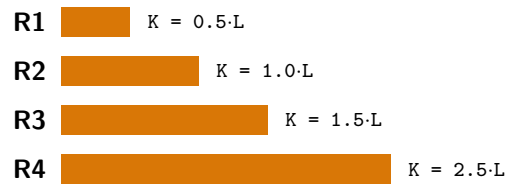
Recap: previous best was pure iteration

`iter_R4_grow_05_10_15_25`

For each of 4 rounds with a growing K:

1. Pick top-K contacts from the previous round's distogram (sum of bins ≤ 8 Å; minimum prob 0.1; ordered long $>$ medium $>$ short).
2. Build prefix:

   `<begin_statements><{range}-range-contact><p_i><p_j>`.
3. Re-read distances at LDDT-shell pairs.



Per-round contact budget. Round 1 picks from the naive baseline distogram.

K per round: $0.5L, 1.0L, 1.5L, 2.5L$.

Result: mean LDDT 0.3511 (+40.7%).
Chain wall: 4373 s (3.16× baseline).

4 / 10 proteins individually pass the +50% bar.
Still 0.023 short of the mean bar.

Why add sampling? Different contacts

Pure iteration picks contacts by the model's **marginal** contact probability — for each pair (i, j) independently, sum $\Pr[d_{ij} < 8\,\text{Å}]$. Pairs the model predicts confidently as contacts.

But the model is autoregressive. **Joint** samples from `<begin_statements> [...]` encode CO-occurring contacts — hints the model wouldn't reveal in marginal reads.

Iteration's contact picks
marginal: top-K pairs by $\Pr[d_{ij} < 8\text{Å}|\ \text{sequence}]$

accurate, deterministic, K-truncated

Sampling's contact picks
joint: pairs that co-occur in autoregressive rollouts from `<begin_statements>`

diverse, stochastic, many per rollout

They are complementary — joint samples help where the marginal is weak, and iteration sharpens what the union proposes.

Sampling, attempt 1: the wrong cutoff

Natural autoregressive generation from `<begin_statements>` emits *any* statement next.
Training docs interleave contacts and distances — `<distance>` is not a "boundary" token.

```
# WRONG: stop at first <distance>
sampling = SamplingParams(temperature=0.7, top_p=0.9, max_tokens=600,
                          stop=["<distance>", "<end>"])
out = llm.generate(prompt, sampling)
# -> emits 2 to 33 contact statements, then a <distance>, stop.
```

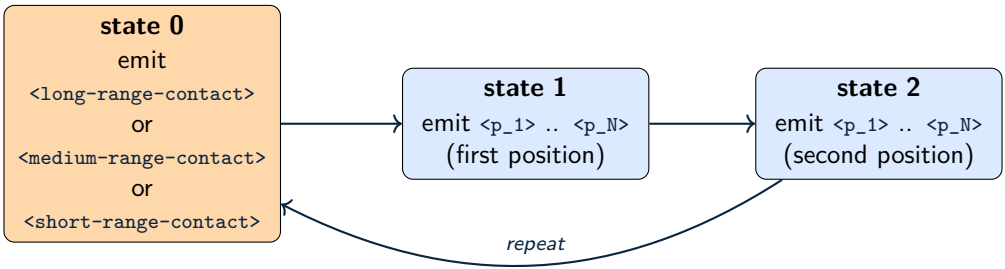
Per-protein contacts harvested per rollout:

protein	8eb9 (L=95)	7y5j (L=102)	7uk8 (L=394)
naive sampling, stop-on-<distance>	2	8	33
(what we actually want)	**100s**	**100s**	**100s**

Result (M=1 average): mean LDDT **0.2568** (+2.9%). Seed set too small.

Sampling, attempt 2: contacts-only LP

Constrain the sampler with a custom `logits_processor` that enforces the grammar of one contact statement, on repeat:



LP receives generated tokens; `state = len(gen) % 3`. All other tokens get $-\infty$ logits. The whole `max_tokens=900` budget is spent emitting contact statements: **225 to 300 unique (i,j) pairs per rollout**.

Surprise: the model's range-token prior is broken

With the LP allowing all 3 range tokens, sample at $T = 0.7$:

protein (L)	long	medium	short
8eb9 (95)	3	222	0
8baq (208)	1	295	0
7uk8 (394)	1	295	0

$\sim 99\%$ **medium-range across every protein.**
The model treats `<medium-range-contact>` as the overwhelmingly likely next-statement token.

But the medium-range pairs aren't necessarily the most informative for LDDT — long-range contacts pin down the global fold, short-range ones lock in local topology. Sampling at the model's natural prior gives 296 medium-range pairs and basically no signal about the rest.

M=1 result with model prior: 0.2219 (-11.1%) — WORSE than baseline.

Is it the prior or the knowledge? (entropy probe)

For each range token r, ask: *after we emit r, how informative is the model's distribution over the next position?*

Measure entropy of `p(next_token | ...<r>)` restricted to position tokens `<p_1>..<p_N>`. Compare to $H_{\max} = \log_2 N$.

protein	$H_{\max}$	H_{long}	H_{med}	H_{short}	gap below max
8eb9 (95)	6.57	5.93	5.96	5.74	0.6–0.8
7y5j (102)	6.67	5.49	5.58	5.09	1.1–1.6
8baq (208)	7.70	6.69	6.68	6.63	1.0–1.1
7uk8 (394)	8.62	6.93	6.84	6.87	1.7–1.8

All 3 ranges have similar (and meaningful) signal. Short tends to be 0.1–0.5 bits sharper. None is "flat".

The medium-bias is in the range-token prior, not in the conditional knowledge.

range_strategy

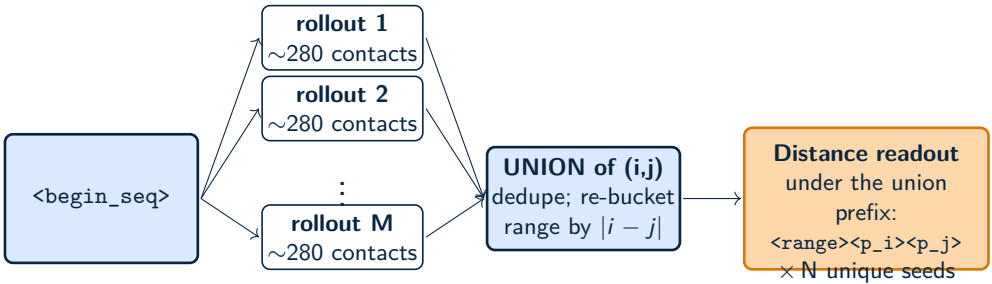
Knob on the LP. At state 0 (next token must be a range token), pick how:

strategy	state-0 behaviour	result (M=1)
`model`	keep model's logits (99% medium)	0.2219 (−11%)
`uniform`	overwrite to $\{0, 0, 0\}$ (1/3 each)	**0.2713 (+8.7%)**
`round_robin`	deterministic L,M,S,L,M,S,...	–

`uniform` implementation — at state 0 the model's prior over the 3 range tokens is ignored:

```
def __call__(generated_token_ids, logits):
    state = len(generated_token_ids) % 3
    if state == 0:
        out = torch.full_like(logits, float("-inf"))
        out[long_id] = 0; out[medium_id] = 0; out[short_id] = 0
        return out
    return logits + position_mask  # state 1 or 2: positions only
```

Pipeline: M independent rollouts, union of contacts



Each rollout uses `logits_processors=[ContactsOnlyLP(strategy="uniform")]`, `T=0.7`, `top_p=0.9`, `max_tokens=900`.

Union of $M = 5$ rollouts gives ∼1500–10000 unique pairs per protein (deduped). The single readout under this rich prefix is the `sampled_uniform_M5_union` distogram.

Sampling alone vs pure iteration: complementary

	baseline	sampled_uniform M=5 union	iter_R4_grow
mean LDDT	0.2496	**0.3142**	**0.3511**
Δ%	–	+25.9%	+40.7%
chain wall (s)	1387	2458	4373

Pure iteration wins overall, *but* per-protein the two gain on DIFFERENT proteins:

protein	baseline	iter_R4_grow	sampled M=5 union
7y5j (L=102)	0.4485	**0.5659**	0.4521
7ykm (L=105)	0.3317	**0.5178**	0.3878
8eb9 (L=95)	0.1510	0.3071	**0.3619 (+0.21!)**
7zs2 (L=316)	0.2500	0.3619	**0.3213**
8cba (L=214)	0.2045	0.2735	**0.2872**

Iteration shines on the model's confident proteins; sampling unlocks the stuck ones. **Combine → next slide.**

The combined algorithm

Stage A: sampled prior

5 independent contacts-only rollouts (LP-constrained, `range_strategy=uniform`)

⇒ union of unique (i,j) seeds

⇒ single distance readout

"what does the joint distribution propose?" — 2458 s

distogram

Stage B: iter R=4 growing K

4 rounds, K per round = 0.5L, 1.0L, 1.5L, 2.5L

pick top-K contacts from prev round's distogram (min prob 0.1)

"sharpen what the marginal endorses" — 3155 s

`Final readout: <begin_seq><AAs><begin_stmts><range><p_i><p_j>...`

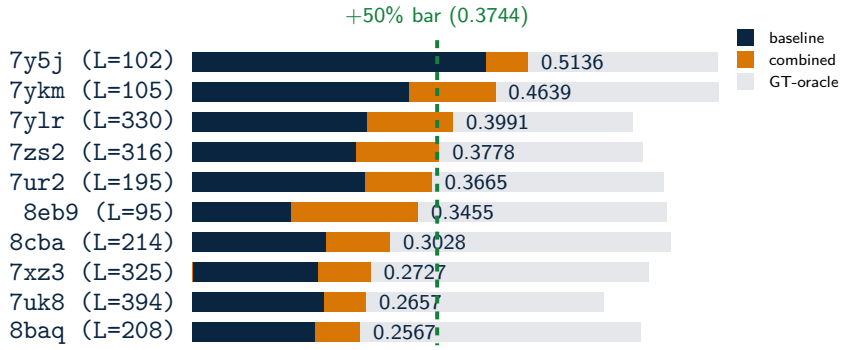
(R4 picks, $\sim 2.5L$ statements; long > med > short ordering)

Standard E[d] = $\sum$ probs · midpoint. No sharpening.

mean LDDT 0.3564 (+42.81%)

chain wall **5613 s** (4.05× baseline; under 5×).

Per-protein result of the combined algorithm



5 / 10 proteins pass the +50% bar individually (up from 4 with iter_R4_grow alone). 7ylr and 7zs2 cross the bar thanks to the sampled prior.

Remaining 5 misses (8baq, 7uk8, 7xz3, 8cba, 8eb9) all gain +0.06 to +0.21 from baseline; their oracle ceilings are 0.62–0.73, so capacity exists but contacts still elude us.

Takeaways

- `<distance>` **is not a boundary.** Training docs interleave contacts and distances; sampling without a grammar constraint stops after ∼10 contacts, leaving the seed set tiny.
- **The model's range-token prior is pathologically biased.** ∼99% medium-range when freely sampled. But all 3 ranges have similar entropy over the next position — **the conditional knowledge is there**.
- **Force uniform range selection at sampling time.** Overwriting the 3 range-token logits to 0 in the LP gives 1/3 of each range; M=1 jumps from −11% to +8.7%.
- **Joint (sampling) and marginal (iteration) contact predictions are complementary.** Sampling unlocks the proteins iteration leaves stuck; iteration sharpens what sampling proposes. Combining them adds +0.005 over iteration alone.
- **Still under the +50% bar by 0.018.** The remaining gap is the model's honest contact-prediction quality on hard proteins, which neither sampling nor iteration can fully extract from this checkpoint at inference time.

Reproducer

10 proteins · A100 40 GB · bf16 · vLLM 0.7.3 · total wall ∼5613 s (4.05× baseline)

```
# 1. Sampled-uniform-M5 union prior (no naive baseline needed)
uv run python run_sampled_contacts.py --dtype bfloat16 --n-gpus 1 \
    --algorithm sampled_uniform_M5_union_T0.7 \
    --n-rollouts 5 --temperature 0.7 --top-p 0.9 --max-sample-tokens 900 \
    --range-strategy uniform --aggregation union
uv run python snapshot_distograms.py --to distogram_sampled_uniform_M5_union.npz

# 2. Iter R=4 grow on top
uv run python run_iterative.py --dtype bfloat16 --n-gpus 1 \
    --algorithm iter_R4_grow_on_sampled_uniform_M5 \
    --n-rounds 4 \
    --k-contacts-per-L-per-round 0.5 1.0 1.5 2.5 \
    --k-distances-per-L-per-round 0.0 0.0 0.0 0.0 \
    --min-contact-prob 0.1 \
    --prior-name distogram_sampled_uniform_M5_union.npz

# Result: mean LDDT 0.3564 (+42.81% over baseline 0.2496)
```

Branch: exp/27-improved-inference-algorithm · Source: sampled_contacts_inference.py, iterative_inference.py